

CS2013 - Programación III

Ejercicios: TDA, copia y movimiento

Clases, lvalues/rvalues y operadores de asignación en C++

Rubén Rivas Medina

Agosto 2026

Alcance

Duración total estimada: 360 minutos. Los seis ejercicios básicos toman aproximadamente 12 minutos cada uno; los intermedios, 18 minutos; y los avanzados, 30 minutos. Use `new[]` y `delete[]` para el recurso principal; no use contenedores de la biblioteca estándar. Todos los identificadores de C++ se presentan en inglés. Los tiempos suponen que el código indicado como “dado” ya fue entregado y que el estudiante conoce `new[]`, `delete[]` y `std::move`; no son tiempos de primera exposición a esos temas.

Los bloques siguientes usan `assert` (`#include <cassert>`). Los observadores mostrados como **given** pertenecen al código base: se incluyen para poder verificar el resultado sin añadir trabajo al ejercicio.

Nivel básico

Ejercicio 1 (Básico · 12 min): IntStack y copia profunda

Contexto. Un evaluador de expresiones necesita conservar una pila de operandos. Antes de evaluar una alternativa, crea una copia de la pila actual; por ello ambos objetos deben poseer arreglos independientes.

El código base ya incluye `push`, `pop` y `top`. Complete constructor, destructor y constructor de copia. Mantenga la invariante $0 \leq \text{size_} \leq \text{capacity_}$; la copia debe reservar su propio arreglo y copiar solo los elementos activos, no toda la capacidad disponible.

```
class IntStack {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit IntStack(std::size_t capacity = 8);
    IntStack(const IntStack& other);
    ~IntStack();
    void push(int value);
    void pop();
    int top() const;
    bool empty() const; // given
};
```

Caso de prueba:

```
IntStack source(4);
source.push(10);
source.push(20);
IntStack copy(source);
copy.pop();
copy.push(99);
assert(source.top() == 20);
assert(copy.top() == 99);
```

Caso adicional:

```

IntStack empty(2);
IntStack copy(empty);
assert(copy.empty());
copy.push(7);
assert(copy.top() == 7);
assert(empty.empty());

```

Ejercicio 2 (Básico · 12 min): GradeBook y asignación por copia

Contexto. Cada docente tiene un libro de calificaciones propio. Al duplicar una sección, las modificaciones del nuevo libro no pueden alterar las notas de la sección original.

Con constructor, destructor y constructor de copia ya dados, implemente solo el operador de asignación. Debe soportar `a = a`, reservar primero la copia nueva y liberar el arreglo anterior solo después de tener una copia válida.

```

class GradeBook {
    double* data_;
    std::size_t size_;
public:
    explicit GradeBook(std::size_t size);
    GradeBook(const GradeBook& other);
    GradeBook& operator=(const GradeBook& other);
    ~GradeBook();
    double& at(std::size_t index);
    std::size_t size() const; // given
};

```

Caso de prueba:

```

GradeBook source(2);
source.at(0) = 15.0;
GradeBook copy(1);
copy = source;
copy.at(0) = 20.0;
copy = copy;
assert(source.at(0) == 15.0);
assert(copy.at(0) == 20.0);

```

Caso adicional:

```

GradeBook empty(0);
GradeBook target(2);
target = empty;
assert(target.size() == 0);
target = target;
assert(target.size() == 0);

```

Ejercicio 3 (Básico · 12 min): IntBuffer con crecimiento

Contexto. Un sensor registra lecturas enteras y no conoce de antemano cuántas recibirá. El búfer empieza pequeño y debe crecer sin perder el orden histórico.

El constructor y destructor están dados. Implemente `resize` y `pushBack`. Cuando se llene el arreglo, duplique su capacidad y copie los enteros en el mismo orden. Después de la realocación, libere exactamente una vez el arreglo anterior y actualice `capacity_` antes de aceptar el nuevo dato.

```

class IntBuffer {
    int* data_;
    std::size_t size_;
    std::size_t capacity_;
    void resize(std::size_t newCapacity);

```

```
public:
    explicit IntBuffer(std::size_t capacity = 2); // given
    void pushBack(int value);
    int at(std::size_t index) const;
    std::size_t size() const; // given
    ~IntBuffer();
};
```

Caso de prueba:

```
IntBuffer buffer(1);
for (int value = 0; value < 5; ++value) buffer.pushBack(value * 10);
assert(buffer.size() == 5);
assert(buffer.at(0) == 0);
assert(buffer.at(4) == 40);
```

Caso adicional:

```
IntBuffer one(1);
one.pushBack(42);
assert(one.size() == 1);
assert(one.at(0) == 42);
```

Ejercicio 4 (Básico · 12 min): CircularQueue y orden lógico

Contexto. Una impresora atiende trabajos en orden FIFO. Cuando el arreglo se envuelve, el primer trabajo lógico ya no está necesariamente en `data_[0]`.

Las operaciones de cola ya están dadas. Implemente solo el constructor de copia; preserve el orden de salida aunque `front_` no sea cero. La copia puede normalizar su `front_` a cero, pero el primer dequeue debe producir el mismo valor que en la cola fuente.

```
class CircularQueue {
    int* data_;
    std::size_t front_;
    std::size_t size_;
    std::size_t capacity_;
public:
    explicit CircularQueue(std::size_t capacity);
    CircularQueue(const CircularQueue& other);
    void enqueue(int value);
    int dequeue();
    bool empty() const; // given
};
```

Caso de prueba:

```
CircularQueue source(3);
source.enqueue(1); source.enqueue(2); source.enqueue(3);
assert(source.dequeue() == 1);
source.enqueue(4);
CircularQueue copy(source);
assert(copy.dequeue() == 2);
assert(copy.dequeue() == 3);
assert(copy.dequeue() == 4);
```

Caso adicional:

```
CircularQueue empty(2);
CircularQueue copy(empty);
```

```
assert(copy.empty());
copy.enqueue(7);
assert(copy.dequeue() == 7);
assert(empty.empty());
```

Ejercicio 5 (Básico · 12 min): Name y copia de char*

Contexto. Una ficha de estudiante guarda su nombre en memoria dinámica. Al generar una ficha de respaldo, las dos fichas no deben compartir el mismo texto.

El constructor, destructor y duplicate están dados. Implemente únicamente el constructor de copia para un nombre almacenado en un único char*. Debe pedir espacio para todos los caracteres más el terminador \0; nunca copie el puntero text_ directamente.

```
class Name {
    char* text_;
    static char* duplicate(const char* text);
public:
    explicit Name(const char* text); // given
    Name(const Name& other);
    ~Name(); // given
    void setFirst(char value); // given
    const char* cStr() const; // given
};
```

Caso de prueba:

```
Name source("Ana");
Name copy(source);
copy.setFirst('L');
assert(std::strcmp(source.cStr(), "Ana") == 0);
assert(std::strcmp(copy.cStr(), "Lna") == 0);
```

Caso adicional:

```
Name empty("");
Name copy(empty);
assert(std::strcmp(empty.cStr(), "") == 0);
assert(std::strcmp(copy.cStr(), "") == 0);
```

Ejercicio 6 (Básico · 12 min): AttendanceRecord y asignación encadenada

Contexto. Un sistema mantiene varias listas de asistencia para diferentes grupos. La asignación debe reemplazar el registro destino sin compartir marcas.

El constructor, destructor y copia están dados. Implemente el operador de asignación para que third = second = first funcione. La función debe devolver AttendanceRecord&, detectar autoasignación y copiar las count_ marcas sin compartir el arreglo present_.

```
class AttendanceRecord {
    bool* present_;
    std::size_t count_;
public:
    explicit AttendanceRecord(std::size_t count);
    AttendanceRecord(const AttendanceRecord& other);
    AttendanceRecord& operator=(const AttendanceRecord& other);
    ~AttendanceRecord();
    void mark(std::size_t student, bool value);
    bool isPresent(std::size_t student) const; // given
};
```

Caso de prueba:

```
AttendanceRecord first(2), second(2), third(2);
first.mark(0, true);
third = second = first;
second = second;
assert(second.isPresent(0));
assert(third.isPresent(0));
```

Caso adicional:

```
second.mark(0, false);
assert(!second.isPresent(0));
assert(first.isPresent(0));
assert(third.isPresent(0));
```

Nivel intermedio

Ejercicio 7 (Intermedio · 18 min): Text y movimiento

Contexto. Un editor temporal construye mensajes que se transfieren a una cola de salida. Mover el mensaje debe transferir su arreglo de caracteres, no duplicarlo, y el objeto origen debe poder destruirse sin problemas.

El constructor, destructor y operaciones de copia están dados. Implemente las dos operaciones de movimiento para transferir el puntero y dejar el origen vacío y válido. La asignación por movimiento debe liberar primero el recurso de `*this`, salvo en auto-movimiento; relacione estos miembros con la regla de cinco.

```
class Text {
    char* chars_;
    std::size_t length_;
public:
    Text(); // given
    explicit Text(const char* source); // given
    ~Text(); // given
    Text(const Text& other); // given
    Text& operator=(const Text& other); // given
    Text(Text&& other) noexcept;
    Text& operator=(Text&& other) noexcept;
    const char* cStr() const; // given
    bool empty() const; // given
};
```

Caso de prueba:

```
Text source("UTEC");
Text moved(std::move(source));
assert(std::strcmp(moved.cStr(), "UTEC") == 0);
assert(source.empty());
Text target("old");
target = std::move(moved);
assert(std::strcmp(target.cStr(), "UTEC") == 0);
assert(moved.empty());
```

Caso adicional:

```
Text empty;
Text copy(std::move(empty));
assert(empty.empty());
assert(copy.empty());
```

Ejercicio 8 (Intermedio · 18 min): IntMatrix y swap

Contexto. Un módulo de cálculo reemplaza matrices temporales con frecuencia. Al mover una matriz, el destino debe adoptar el bloque de datos sin copiar cada celda, y el origen debe quedar con dimensiones cero.

El constructor, destructor y copia ya están dados. Implemente `swap`, el constructor de movimiento y la asignación por movimiento. `swap` debe intercambiar únicamente el puntero y las dimensiones; no copie los valores de la matriz. Tras mover, el origen debe tener dimensiones cero.

```
class IntMatrix {
    int* data_;
    std::size_t rows_;
    std::size_t columns_;
    void swap(IntMatrix& other) noexcept;
public:
    IntMatrix(std::size_t rows, std::size_t columns);
    IntMatrix(IntMatrix&& other) noexcept;
    IntMatrix& operator=(IntMatrix&& other) noexcept;
    std::size_t rows() const; // given
```

```
std::size_t columns() const; // given
};
```

Caso de prueba:

```
IntMatrix source(2, 3);
IntMatrix target(1, 1);
target = std::move(source);
assert(target.rows() == 2 && target.columns() == 3);
assert(source.rows() == 0 && source.columns() == 0);
```

Caso adicional:

```
IntMatrix empty(0, 0);
IntMatrix moved(std::move(empty));
assert(moved.rows() == 0 && moved.columns() == 0);
assert(empty.rows() == 0 && empty.columns() == 0);
```

Ejercicio 9 (Intermedio · 18 min): Polynomial y retorno por valor

Contexto. Un módulo simbólico necesita derivar polinomios para calcular velocidades. La derivada es un objeto nuevo; el polinomio de entrada representa la ecuación original y no debe cambiar.

El manejo del arreglo ya está implementado. Programe `derivative`, que crea y devuelve un nuevo polinomio sin modificar el objeto original. Si $p(x) = a_0 + a_1x + \dots + a_nx^n$, el coeficiente $i - 1$ del resultado debe ser $i * a_i$. Trate el polinomio constante como un caso especial válido.

```
class Polynomial {
    double* coefficients_;
    std::size_t degree_;
public:
    explicit Polynomial(std::size_t degree);
    Polynomial derivative() const;
    double evaluate(double x) const;
    double& coefficient(std::size_t exponent); // given
};
```

Caso de prueba:

```
Polynomial source(2); // 3x^2 + 2x + 1
source.coefficient(0) = 1;
source.coefficient(1) = 2;
source.coefficient(2) = 3;
Polynomial derived = source.derivative();
assert(source.evaluate(2) == 17);
assert(derived.evaluate(2) == 14); // 6x + 2
```

Caso adicional:

```
Polynomial constant(0);
constant.coefficient(0) = 8;
Polynomial zero = constant.derivative();
assert(zero.evaluate(0) == 0);
assert(zero.evaluate(5) == 0);
```

Ejercicio 10 (Intermedio · 18 min): GrayImage y objeto temporal

Contexto. Un filtro de imágenes produce versiones temporales antes de mostrarlas. El filtro `inverted` debe crear una imagen distinta y los resultados temporales deben poder moverse eficientemente.

El constructor, destructor y copia están dados. Implemente el constructor de movimiento, la asignación por movimiento e inverted. Para cada píxel p, la imagen invertida debe almacenar $255 - p$; el método no puede alterar la imagen original y debe devolver un objeto independiente.

```
class GrayImage {
    unsigned char* pixels_;
    std::size_t width_;
    std::size_t height_;
public:
    GrayImage(std::size_t width, std::size_t height);
    GrayImage(GrayImage&& other) noexcept;
    GrayImage& operator=(GrayImage&& other) noexcept;
    GrayImage inverted() const;
    unsigned char& at(std::size_t row, std::size_t column);
    unsigned char at(std::size_t row, std::size_t column) const; // given
    bool empty() const; // given
};
```

Caso de prueba:

```
GrayImage source(1, 1);
source.at(0, 0) = 10;
GrayImage result = source.inverted();
assert(source.at(0, 0) == 10);
assert(result.at(0, 0) == 245);
GrayImage target(1, 1);
target = std::move(result);
assert(target.at(0, 0) == 245 && result.empty());
```

Caso adicional:

```
GrayImage white(1, 1);
white.at(0, 0) = 255;
GrayImage black = white.inverted();
assert(white.at(0, 0) == 255);
assert(black.at(0, 0) == 0);
```

Ejercicio 11 (Intermedio · 18 min): ScoreList y movimiento explícito

Contexto. Un generador arma una lista de puntajes de muestra y la devuelve al programa principal. La devolución por valor permite observar elisión de copia o movimiento sin modificar la lista recibida.

Con la regla de cinco ya implementada, agregue makeSample, que devuelve una lista por valor. Inserte tres valores dentro de esa función e instrumente las operaciones de movimiento con un mensaje para identificar si hay elisión de copia o movimiento al inicializar el receptor.

```
class ScoreList {
    int* scores_;
    std::size_t size_;
    std::size_t capacity_;
public:
    ScoreList(); // given
    void add(int score);
    std::size_t size() const; // given
};
```

Caso de prueba:

```
ScoreList makeSample() {
    ScoreList result;
    result.add(10);
```



```

    result.add(20);
    return result;
}
ScoreList scores = makeSample();
assert(scores.size() == 2);

```

Caso adicional:

```

void consume(ScoreList value) {
    assert(value.size() == 2);
}
consume(makeSample());

```

Ejercicio 12 (Intermedio · 18 min): CourseRoster y copia segura

Contexto. Un curso puede tener una lista de identificadores vacía o una lista activa. Si falla la reserva de memoria al copiar, el roster destino debe conservar su estado anterior; swap permite organizar esa operación.

El constructor, destructor y copia están dados. Implemente swap y la asignación por copia con una copia temporal. El temporal debe adquirir la copia de other; luego swap deja a *this actualizado y el temporal destruye el arreglo antiguo al salir del operador.

```

class CourseRoster {
    int* ids_;
    std::size_t size_;
    void swap(CourseRoster& other) noexcept;
public:
    explicit CourseRoster(std::size_t size = 0);
    CourseRoster& operator=(const CourseRoster& other);
    std::size_t size() const; // given
};

```

Caso de prueba:

```

CourseRoster empty;
CourseRoster copy(3);
copy = empty;
assert(copy.size() == 0);
copy = copy;
assert(copy.size() == 0);

```

Caso adicional:

```

CourseRoster source(4);
CourseRoster target(2);
target = source;
assert(target.size() == 4);
assert(source.size() == 4);

```

Nivel avanzado

Ejercicio 13 (Avanzado · 30 min): History para lvalue y rvalue

Contexto. Un registro de eventos puede recibir un mensaje que continuará usándose fuera del historial o un mensaje temporal que ya no se necesita. La interfaz debe copiar en el primer caso y mover en el segundo.

Use Text como elemento y un arreglo dinámico como almacenamiento. La regla de cinco y el crecimiento están dados; defina las dos sobrecargas de add. add(const Text&) debe conservar el mensaje del llamador; add(Text&&) debe usar std::move para construir la nueva entrada. Ambas deben incrementar size_ exactamente una vez.

```
class History {
    Text* entries_;
    std::size_t size_;
    std::size_t capacity_;
public:
    History(); // given
    void add(const Text& entry);
    void add(Text&& entry);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};
```

Caso de prueba:

```
History history;
Text message("start");
history.add(message);
history.add(Text("stop"));
history.add(std::move(message));
assert(history.size() == 3);
assert(std::strcmp(history.at(0).cStr(), "start") == 0);
assert(std::strcmp(history.at(1).cStr(), "stop") == 0);
```

Caso adicional:

```
Text reusable("keep");
history.add(reusable);
assert(std::strcmp(reusable.cStr(), "keep") == 0);
assert(std::strcmp(history.at(3).cStr(), "keep") == 0);
```

Ejercicio 14 (Avanzado · 30 min): TaskList y separación de resultados

Contexto. Una aplicación separa al cierre del día las tareas completadas de las pendientes. La operación debe crear una segunda lista sin reordenar las tareas que permanecen ni las que se extraen.

La regla de cinco, add y complete están dadas. Implemente extractCompleted: devuelve las terminadas y las elimina del original, preservando el orden. Recorra las tareas una sola vez para contar o clasificar; la lista devuelta y la lista original deben poseer arreglos distintos.

```
struct Task {
    Text title;
    bool completed;
};
class TaskList {
    Task* tasks_;
    std::size_t size_;
    std::size_t capacity_;
public:
    TaskList(); // given
    TaskList extractCompleted();
    void add(Text&& title); // given
```

```

void complete(std::size_t index); // given
std::size_t size() const; // given
const Task& at(std::size_t index) const; // given
};

```

Caso de prueba:

```

TaskList pending;
pending.add(Text("code"));
pending.add(Text("test"));
pending.complete(1);
TaskList done = pending.extractCompleted();
assert(pending.size() == 1 && done.size() == 1);
assert(std::strcmp(done.at(0).title.cStr(), "test") == 0);

```

Caso adicional:

```

TaskList pendingOnly;
pendingOnly.add(Text("read"));
TaskList none = pendingOnly.extractCompleted();
assert(pendingOnly.size() == 1);
assert(none.size() == 0);

```

Ejercicio 15 (Avanzado · 30 min): TextList y realocación con movimiento

Contexto. Un chat almacena mensajes y debe crecer cuando llega más texto del esperado. Durante la realocación, los objetos Text ya existentes deben ser movidos al nuevo arreglo para evitar duplicar todos sus caracteres.

La regla de cinco y la sobrecarga para lvalue están dadas. Implemente el crecimiento y add(Text&&). Al crecer, traslade los elementos con std::move. El nuevo arreglo debe tener la capacidad pedida y, tras mover cada elemento, debe liberarse el arreglo antiguo. La sobrecarga rvalue no debe crear una copia intermedia del argumento.

```

class TextList {
    Text* data_;
    std::size_t size_;
    std::size_t capacity_;
    void resize(std::size_t newCapacity);
public:
    explicit TextList(std::size_t capacity = 2); // given
    void add(const Text& value); // given
    void add(Text&& value);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};

```

Caso de prueba:

```

TextList list(1);
Text first("one");
list.add(first);
list.add(Text("two"));
list.add(Text("three"));
assert(list.size() == 3);
assert(std::strcmp(list.at(2).cStr(), "three") == 0);

```

Caso adicional:

```
Text external("outside");
TextList check(1);
check.add(external);
check.add(Text("next"));
assert(std::strcmp(external.cStr(), "outside") == 0);
assert(std::strcmp(check.at(0).cStr(), "outside") == 0);
```

Ejercicio 16 (Avanzado · 30 min): AdjacencyMatrix y transposición

Contexto. Un grafo dirigido describe llamadas entre servicios. Para descubrir quién llama a un servicio, se usa el grafo transpuesto, donde cada arista $u \rightarrow v$ pasa a ser $v \rightarrow u$.

Use un grafo dirigido pequeño con un arreglo dinámico de `bool`; el manejo del recurso está dado. Implemente `transposed`, que devuelve las aristas invertidas. Para cada posición (`from`, `to`) verdadera, la posición (`to`, `from`) del nuevo grafo debe ser verdadera. No modifique `edges_` del objeto fuente.

```
class AdjacencyMatrix {
    bool* edges_;
    std::size_t vertices_;
public:
    explicit AdjacencyMatrix(std::size_t vertices);
    void connect(std::size_t from, std::size_t to);
    AdjacencyMatrix transposed() const;
    bool connected(std::size_t from, std::size_t to) const; // given
};
```

Caso de prueba:

```
AdjacencyMatrix graph(3);
graph.connect(0, 2);
AdjacencyMatrix transposed = graph.transposed();
assert(graph.connected(0, 2));
assert(!graph.connected(2, 0));
assert(transposed.connected(2, 0));
```

Caso adicional:

```
AdjacencyMatrix empty(2);
AdjacencyMatrix reversed = empty.transposed();
assert(!reversed.connected(0, 1));
assert(!reversed.connected(1, 0));
```

Ejercicio 17 (Avanzado · 30 min): FileBuffer como tipo solo movable

Contexto. Un búfer representa el contenido temporal de un archivo grande. Solo debe tener un dueño; permitir copias accidentales duplicaría memoria y haría ambigua la responsabilidad de liberar el recurso.

Modele un búfer de bytes de propietario único. El constructor y destructor están dados; elimine las operaciones de copia e implemente las dos de movimiento. No realice lectura de archivos. El movimiento debe transferir `bytes_`, copiar `size_` al destino y reiniciar el origen con `nullptr` y tamaño cero. La asignación por movimiento debe liberar primero el búfer que ya posee el destino.

```
class FileBuffer {
    char* bytes_;
    std::size_t size_;
public:
    explicit FileBuffer(std::size_t size); // given
    FileBuffer(const FileBuffer&) = delete;
    FileBuffer& operator=(const FileBuffer&) = delete;
    FileBuffer(FileBuffer&& other) noexcept;
    FileBuffer& operator=(FileBuffer&& other) noexcept;
    ~FileBuffer(); // given
```

```
std::size_t size() const; // given
};
```

Caso de prueba:

```
FileBuffer source(128);
FileBuffer target(std::move(source));
assert(target.size() == 128);
assert(source.size() == 0);
target = std::move(target); // auto-movimiento defensivo
assert(target.size() == 128);
```

Caso adicional:

```
FileBuffer replacement(64);
target = std::move(replacement);
assert(target.size() == 64);
assert(replacement.size() == 0);
```

Ejercicio 18 (Avanzado · 30 min): DocumentIndex y fusión por movimiento

Contexto. Dos documentos indexan palabras por separado y luego se consolidan. La fusión recibe un índice temporal y transfiere sus entradas al destino, sin dejar al origen con punteros que ya no posee.

La regla de cinco y add están dados. Implemente merge, que toma un rvalue, crece solo si es necesario e incorpora sus entradas. El origen queda vacío y válido. Preserve primero las palabras ya existentes en *this; luego traslade en orden las entradas de other y actualice ambos tamaños. Debe funcionar si other está vacío.

```
class DocumentIndex {
    Text* words_;
    std::size_t size_;
    std::size_t capacity_;
public:
    DocumentIndex(); // given
    void add(Text&& word); // given
    void merge(DocumentIndex&& other);
    std::size_t size() const; // given
    const Text& at(std::size_t index) const; // given
};
```

Caso de prueba:

```
DocumentIndex first, second;
first.add(Text("cat"));
second.add(Text("dog"));
first.merge(std::move(second));
assert(first.size() == 2);
assert(second.size() == 0);
assert(std::strcmp(first.at(1).cStr(), "dog") == 0);
```

Caso adicional:

```
DocumentIndex empty;
std::size_t oldSize = first.size();
first.merge(std::move(empty));
assert(first.size() == oldSize);
assert(empty.size() == 0);
```

Lista de verificación

- Cada clase libera exactamente el recurso que posee.

- Las copias son profundas y la autoasignación es segura.
- Los movimientos dejan el origen válido y destruible.
- Las pruebas incluyen al menos un lvalue y un rvalue cuando el ejercicio lo requiere.